

**FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA
ENGENHARIA DE SOFTWARE**

Guilherme Dal Posolo Matheus — RM98694

Guilherme Faustino Vargas — RM98278

João Lucas Yudi Redi Handa — RM98458

Lucas Laia Manentti — RM97709

Ryan Perez Pacheco — RM98782

**CLASSIFICAÇÃO DE ÁREAS PROPENSAS A QUEIMADAS VIA IMAGENS DE
SATÉLITE.**

APPLIED COMPUTER VISION - GLOBAL SOLUTION 2026

4ESPV

SÃO PAULO

2026

Resumo

Este trabalho apresenta uma solução de visão computacional ao tema da Global Solution 2026 - Space Connect, que foca no monitoramento ambiental por sensoriamento remoto. Nós treinamos duas redes neurais do zero para que fosse possível classificar imagens de satélite em duas classes: áreas com histórico de queimadas (wildfire) e áreas sem histórico de queimadas (nowildfire). O modelo mais simples, usado como baseline, atingiu 73,79% de precisão mas colapsou para prever apenas a classe majoritária. O modelo mais preciso, com Batch Normalization, Dropout, Global Average Pooling e Data Augmentation, atingiu 97,30% de precisão no conjunto de testes, o qual superou o critério mínimo de 88%. O projeto foi entregue junto com uma aplicação Streamlit básica para uma melhor interatividade do modelo.

1. Definição do problema

As queimadas florestais geram grande impacto no clima, na saúde pública e na biodiversidade. No Brasil, o problema é muito relevante devido à Amazônia, ao Cerrado e ao Pantanal. Como o combate ao fogo após ser iniciado é algo relativamente caro e nem sempre eficiente, surge a necessidade de identificar áreas de risco antes de ocorrer o acidente.

Junto ao tema da Global Solution de 2026, "Space Connect", este projeto utiliza imagens de satélite para classificar lugares propensos a queimadas. O modelo faz uma classificação binária, analisando características da paisagem, como vegetação, uso do solo e relevo, associadas ao histórico de ocorrência destes eventos.

O projeto não detecta focos de incêndios ativos, mas auxilia na prevenção, permitindo que órgãos responsáveis direcionam ações de monitoramento e forte fiscalização para áreas mais vulneráveis.

2. Dataset

2.1 Origem

Foi usado o Wildfire Prediction Dataset (Satellite Images), disponibilizado no Kaggle pelo autor Abdelghani Aaba. O dataset foi criado a partir de coordenadas geográficas de queimadas que foram registradas oficialmente pelo governo do Canadá. Em cada local com ocorrência registrada, foi obtida uma imagem de satélite via API do MapBox que é atribuída à classe wildfire. Já para coordenadas sem registros de queimadas, as imagens foram rotuladas como nowildfire.

Por este motivo, os arquivos possuem nomes formados pelas coordenadas, como -73.602345,45.411799.jpg. As imagens que foram utilizadas são exclusivamente imagens de satélite, representando vistas aéreas das regiões analisadas.

2.2 Estatísticas

O dataset foi disponibilizado pelo autor com divisões de treino, validação e teste, aproximadamente. Essa separação foi mantida por apresentar boa distribuição entre as classes, o conjunto é praticamente balanceado, com cerca de 53% de imagens wildfire e 47% nowildfire.

2.3 Características técnicas

Todas as imagens tem resolução 350x350 pixels em formato JPEG, com 3 canais RGB. A consistência desse formato facilitou bastante o pré-processamento, pois não precisamos lidar com imagens de tamanhos ou formatos diferentes.

2.4 Pré-processamento e limpeza

Durante o treinamento inicial, nós encontramos alguns arquivos JPEG corrompidos no dataset. O TensorFlow quebra com um erro de decodificação quando isso acontece. Então nós implementamos uma rotina de verificação que abre cada imagem com a biblioteca PIL, valida a integridade do header e remove os arquivos inválidos. Foram identificados poucos arquivos com problemas e removidos antes do treinamento definitivo.

No momento do carregamento as imagens foram redimensionadas para 128x128 pixels, pois reduzi-las de 350x350 para 128x128 acelera o treinamento de forma significativa sem perda perceptível de precisão para esse problema.

A normalização foi feita de formas um pouco diferentes nos dois modelos: no baseline, aplicada no pipeline `tf.data`; no modelo avançado, embutida como uma camada Rescaling dentro do próprio modelo.

3. Arquitetura das redes

A proposta apresentada no trabalho pede no mínimo duas CNNs treinadas do zero, sem uso de modelos pré-treinados. Nós optamos por duas abordagens contrastantes para conseguir uma boa comparação técnica.

3.1 Modelo Baseline

A primeira rede foi pensada como uma CNN simples, sem nenhuma técnica de regularização. A ideia era estabelecer um ponto de partida e observar o overfitting clássico, que iria justificar as melhorias do segundo modelo.

Estrutura:

Input (128x128x3)

- Conv2D(32, 3x3, ReLU)
- MaxPool(2x2)
- Conv2D(64, 3x3, ReLU)
- MaxPool(2x2) -> Conv2D(128, 3x3, ReLU)
- MaxPool(2x2) -> Flatten
- Dense(128, ReLU)
- Dense(1, Sigmoid)

Total de parâmetros: 4.287.809. A maioria desses parâmetros está concentrada na camada Flatten seguida do Dense(128), que é onde está o gargalo de parâmetros desse tipo de arquitetura.

Compilação:

- Otimizador: Adam, learning rate 0,001
- Função de perda: binary_crossentropy
- Métrica: accuracy

3.2 Modelo Avançado

A segunda rede neural usa várias técnicas que hoje são consideradas boas práticas em CNNs para classificação de imagens:

1. **BatchNormalization** após cada Conv2D - normaliza as ativações entre camadas, o que estabiliza os gradientes e permite treinos mais profundos sem explosão de loss.
2. **Dropout progressivo** (0,2 - 0,25 - 0,3 - 0,35 - 0,5) - desliga neurônios de forma aleatória durante o treino, forçando a rede a não depender de features específicas. O dropout maior na camada densa final é proposital, porque é onde o overfitting tende a se concentrar.
3. **Global Average Pooling** ao invés de Flatten - reduz o número de parâmetros densos. Em vez de "achatar" o tensor 8x8x256 inteiro, faz a média de cada canal e gera só 256 valores.
4. **Data Augmentation** dentro do modelo - flip horizontal/vertical, rotação aleatória de até 20%, zoom de até 15% e variação de contraste de até 10%. Isso amplia de forma virtual o dataset e faz o modelo aprender padrões mais robustos.
5. **Rescaling** como camada do modelo - a normalização vira uma parte do modelo, então ele já carrega "pronto para usar".

6. Padrão Conv - BN - Activation com `use_bias=False` nas Conv2D - como a BN já tem um parâmetro de viés, o bias da Conv vira redundante.

4. Treinamento

4.1 Configuração

Os modelos foram treinados em um MacBook Air M4, utilizando aceleração via GPU integrada (Metal). Foi preciso usar versões específicas do ambiente devido a incompatibilidade das versões mais recentes.

O treinamento usou batch size 32, adequado ao tamanho do dataset e a memória disponível.

No modelo avançado, foram usadas estratégias para melhorar o treinamento e evitar perda de desempenho. O sistema salva automaticamente a melhor versão do modelo, interrompe o treinamento quando não há mais melhorias e ajusta a velocidade de aprendizado para facilitar a convergência dos resultados. Já o modelo baseline usou apenas os mecanismos de salvamento da melhor versão e interrupção antecipada do treinamento.

4.2 Curvas de aprendizado

Os dois modelos tiveram comportamentos bem diferentes durante o treinamento. O baseline apresentou resultados bastante instáveis durante o treinamento, grandes variações nos indicadores de desempenho entre as épocas. Em alguns momentos, o desempenho melhorou significativamente e, em seguida, piorava sem um padrão, indicando dificuldades no processo de aprendizado.

O modelo avançado apresentou um comportamento muito mais estável. Seu desempenho melhorou de forma gradual ao longo do treinamento, alcançando resultados melhores. Nas partes finais surgiu um leve sinal de excesso de ajuste aos dados de treino, mas o sistema recuperou automaticamente a versão que havia conseguido o melhor desempenho, garantindo uma melhor capacidade de generalização.

5. Resultados

Métricas no conjunto de teste

MÉTRICA	BASELINE	AVANÇADO
---------	----------	----------

Acurácia	73,79%	97,30%
Loss	Alto, instável	0,0701

Matrizes de confusão

Modelo Baseline:

	PREDITO: NOWILDFIRE	PREDITO: WILDFIRE
REAL: NOWILDFIRE	0	2.820
REAL: WILDFIRE	0	3.479

Modelo Avançado:

	PREDITO: NOWILDFIRE	PREDITO: WILDFIRE
REAL: NOWILDFIRE	2.748	72
REAL: WILDFIRE	98	3.381

6. Demonstração funcional

Para demonstração da solução e melhor visualização, nós desenvolvemos uma simples aplicação web utilizando Streamlit. A interface permite que o usuário faça upload de uma imagem satelital e recebe no mesmo momento uma classificação, junto com a probabilidade associada e uma barra visual mostrando a distribuição entre as duas classes.

A aplicação carrega o modelo avançado, que é o melhor modelo treinado automaticamente na primeira execução e mantém em cache para previsões subsequentes. Cada previsão roda em fração de segundos, mesmo em CPU.

6.1 Observações dos testes:

- Em imagens claramente “wildfire” ou claramente “nowildfire” do conjunto de teste, o modelo costuma prever com confiança acima de 95%
- Em imagens com características ambíguas, a confiança cai para a faixa de 60%-80%. Isso é, na verdade, um comportamento desejável: o modelo está sendo honesto sobre sua incerteza, e não apenas chutando alto.

- Para uso real, faria sentido definir um threshold de confiança abaixo do qual o sistema sinalizaria que precisa de revisão humana, ao invés de tomar uma decisão automática.

A aplicação é executada com o comando (streamlit run app/[app.py](#)) a partir da raiz do projeto, com o ambiente virtual ativado.

7. Conclusão

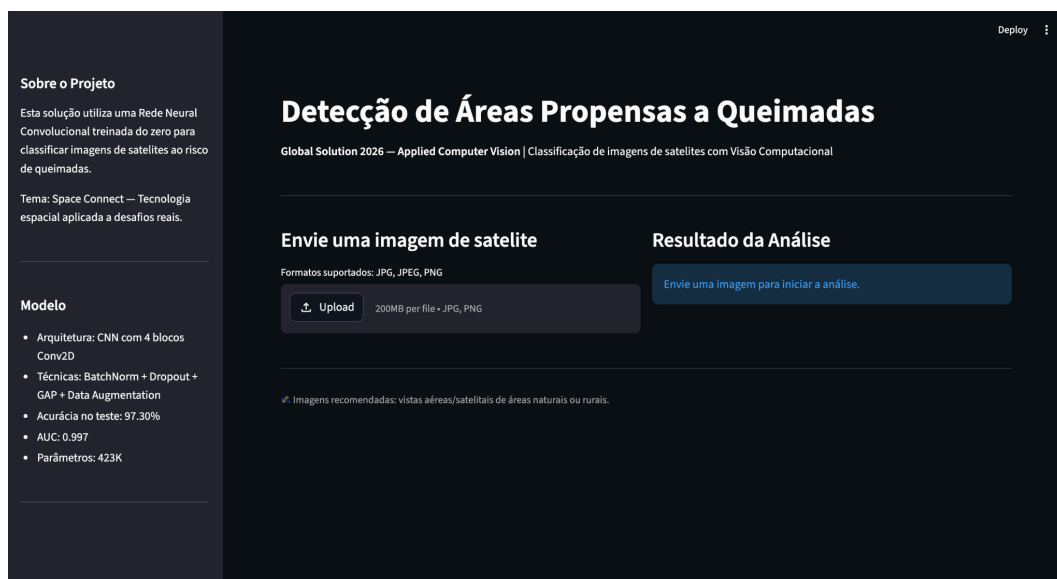
O projeto atingiu os objetivos propostos. Foram usadas duas redes neurais em TensorFlow/Keras, comparadas entre si, e o modelo final alcançou 97,30% de precisão nos testes, superando o mínimo de 88% estabelecido.

Durante o desenvolvimento, obtivemos dois aprendizados destacados. O primeiro é que a acurácia, de forma isolada, não é suficiente para avaliar um modelo, sendo necessário analisar outras métricas para verificar seu desempenho real. O segundo é que modelos maiores nem sempre são melhores: o modelo avançado, mesmo com menos parâmetros, apresentou resultados superiores.

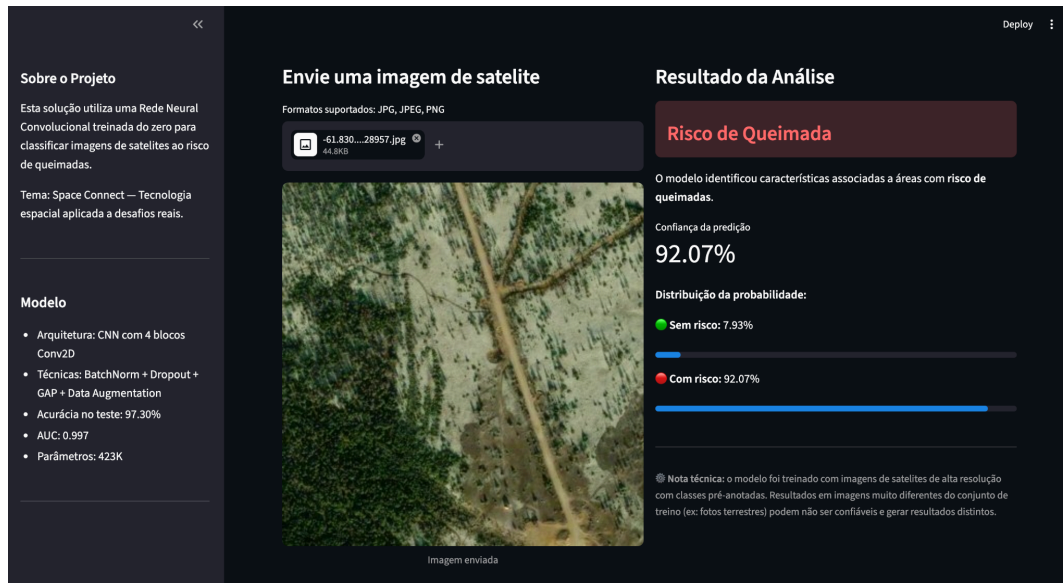
Em relação ao tema Space Connect, a solução demonstra o potencial do uso de imagens de satélite para ajudar em ações preventivas de monitoramento ambiental. A aplicação desenvolvida funciona como uma prova de conceito, mostrando como essa tecnologia pode ajudar profissionais e órgãos responsáveis pela prevenção de queimadas.

Imagens da aplicação Streamlit funcionando

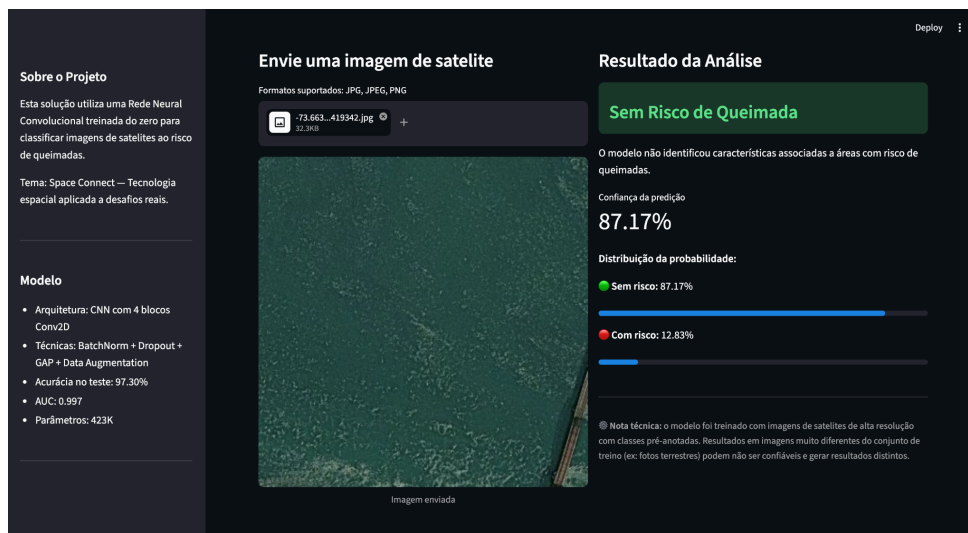
Tela principal:



Wildfire:



Nowildfire:



Como rodar o projeto

Pré-requisitos:

- MacOS, Linux ou Windows
- Python 3.12 (versões mais recentes podem não ser compatíveis com o TensorFlow + Metal no Apple Silicon)
- Git (para clonar o repositório)
- Aproximadamente 3GB de espaço em disco
- Conta no Kaggle para download do dataset.

As imagens do dataset que podem ser usadas para testar o programa estão na pasta:

- Dados/Teste/wildfire
- Dados/Teste/nowildfire

Recomendação para testes:

1. Pelo menos uma imagem de cada classe (wildfire e nowildfire) - para verificar predição correta.
2. Uma imagem ambígua - para observar o comportamento do modelo em casos de fronteira (confiança intermediária entre 60%-80%)
3. Opcionalmente, uma imagem de satélite externa (exemplo: do Google Earth) - importante destacar que o modelo foi treinado com imagens do Canadá, então imagens de outras regiões podem apresentar comportamento menos confiável.

Link repositório:

<https://github.com/guivargas2004/global-solution-2026-queimadas>

Referências

Dataset original: Aaba, Abdelghani. *Wildfire Prediction Dataset (Satellite Images)*.

Kaggle, disponível em

<https://www.kaggle.com/datasets/abdelghaniaaba/wildfire-prediction-dataset>

TensorFlow / Keras: documentação oficial em <https://www.tensorflow.org>

Streamlit: documentação oficial em <https://streamlit.io>

Apple Metal Plugin para TensorFlow:

<https://developer.apple.com/metal/tensorflow-plugin/>

Ioffe, S.; Szegedy, C. *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. ICML 2015.

Srivastava, N. et al. *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*. JMLR 2014.

Lin, M.; Chen, Q.; Yan, S. *Network In Network*. ICLR 2014. (artigo original do Global Average Pooling).